🛠️ How I Built the DevOps Todo API — Step by Step

This document walks through every decision and step taken to build this project from scratch — from writing the app to containerizing it, automating the CI/CD pipeline, and deploying it to Kubernetes.

Table of Contents

1. Project Goal
2. Step 1 — Designing the Folder Structure
3. Step 2 — Building the Node.js Application
4. Step 3 — Writing Tests
5. Step 4 — Dockerizing the Application
6. Step 5 — Docker Compose for Local Development
7. Step 6 — Setting Up the Jenkins CI/CD Pipeline
8. Step 7 — Kubernetes Manifests
9. Step 8 — Kustomize Overlays (Dev vs Prod)
10. Step 9 — Helper Scripts
11. Key DevOps Concepts Applied
12. Tech Stack Summary

---

1. Project Goal

The goal was to build a **full DevOps project** that demonstrates real-world usage of:

- **Docker** — containerize the app with a secure, multi-stage build
- **Jenkins** — automate the full CI/CD lifecycle (test → build → push → deploy)
- **Kubernetes** — orchestrate the containers at scale with zero-downtime deployments

The application itself is a simple **Todo REST API** built with Node.js and Express. The app is intentionally simple so the focus stays on the DevOps tooling around it.

---

2. Step 1 — Designing the Folder Structure

Before writing any code, the folder structure was planned to keep concerns cleanly separated:

```
devops-project/
├── app/            ← Application code and tests
├── docker/         ← Nginx config
├── jenkins/        ← Jenkins config as code
├── k8s/
│   ├── base/       ← Shared Kubernetes manifests
│   └── overlays/   ← Environment-specific overrides (dev / prod)
├── scripts/        ← Bash automation scripts
├── Dockerfile      ← Multi-stage Docker build
├── docker-compose.yml
├── Jenkinsfile     ← CI/CD pipeline definition
└── README.md
```

Why this structure? Separating `base` and `overlays` in Kubernetes follows the **Kustomize** pattern — write your manifests once and patch them per environment without duplication. Everything else is grouped by tool so the repo is easy to navigate.

---

3. Step 2 — Building the Node.js Application

File: `app/src/index.js`

The app was built using **Express.js** with the following middleware and features:

Middleware	Purpose
`helmet`	Sets secure HTTP headers automatically
`cors`	Allows cross-origin requests
`morgan`	HTTP request logging
`express.json()`	Parses incoming JSON bodies

API Endpoints

Method	Route	Description
GET	`/health`	Returns app status + version (used by Kubernetes liveness probe)
GET	`/ready`	Readiness check (used by Kubernetes readiness probe)
GET	`/api/todos`	List all todos
GET	`/api/todos/:id`	Get a single todo

Method	Route	Description
POST	`/api/todos`	Create a new todo
PUT	`/api/todos/:id`	Update a todo
DELETE	`/api/todos/:id`	Delete a todo

Two endpoints — `/health` and `/ready` — were added specifically for Kubernetes health probes. This is a production best practice: the liveness probe restarts an unhealthy container, while the readiness probe removes it from the load balancer until it is ready to serve traffic.

---

4. Step 3 — Writing Tests

File: `app/tests/app.test.js`

Tests were written with **Jest** and **Supertest**. Supertest lets you make HTTP requests to the Express app in-memory without spinning up a real server, making tests fast and self-contained.

8 tests were written covering:

- Health and readiness endpoints returning correct status codes
- Listing all todos
- Creating a todo (happy path and missing-title validation)
- Getting a specific todo (found and not found)
- Updating a todo
- Deleting a todo

The `package.json` defines two test scripts:

- `npm test` — for local development
- `npm run test:ci` — adds the `--ci` flag for Jenkins (stricter, no watch mode)

Coverage output goes to `app/coverage/` which Jenkins later publishes as an HTML report.

---

5. Step 4 — Dockerizing the Application

File: `Dockerfile`

A **multi-stage Dockerfile** was used with three stages:

Stage 1 — `deps`

Installs all npm dependencies. By copying only `package*.json` before the source code, Docker

caches this layer. If only source files change on the next build, npm install is skipped entirely — saving significant build time.

Stage 2 — `test`

Copies the full app and runs `npm run test:ci`. If any test fails, the Docker build itself fails at this stage. This means **you literally cannot build a broken image** — the test failure blocks it.

Stage 3 — `production`

The final image:

- Uses only production dependencies (no test tooling)
- Creates a non-root user (`appuser`) and runs as that user — a security best practice
- Drops all Linux capabilities
- Marks the filesystem as read-only
- Exposes port 3000
- Includes a `HEALTHCHECK` instruction so Docker itself can monitor the container

The result is a lean, secure image where the attack surface is minimized.

---

6. Step 5 — Docker Compose for Local Development

File: `docker-compose.yml`

Docker Compose wires together three services for local development:

`app` (Node.js API)

Built from the `Dockerfile` using the `production` target. Runs on port `3000`.

`jenkins` (CI/CD server)

Uses the official `jenkins/jenkins:lts-jdk17` image. The Docker socket (`/var/run/docker.sock`) is mounted into the Jenkins container so Jenkins can run Docker commands — this is called **Docker-in-Docker (DinD)**. Jenkins' home directory is stored in a named volume so data persists across restarts.

`nginx` (Reverse Proxy)

A lightweight Nginx container sits in front of the app. It handles:

- Routing requests to the app container by name (`app:3000`)
- **Rate limiting** — limits each client to 30 requests per minute to prevent abuse
- Forwarding real client IP headers

---

7. Step 6 — Setting Up the Jenkins CI/CD Pipeline

File: `Jenkinsfile`

The pipeline is written as a **Declarative Pipeline** — a structured, readable DSL for Jenkins. It has 9 stages:

Stage 1 — Checkout

Clones the repository and captures git metadata (author name, commit message) which can be used in notifications.

Stage 2 — Install

Runs `npm ci` instead of `npm install`. The `ci` command is stricter — it reads the `package-lock.json` exactly, making builds reproducible.

Stage 3 — Lint

A quick syntax check on the source file. In a full project, this is where ESLint would run.

Stage 4 — Test

Runs the Jest test suite with coverage. The stage is wrapped in a `when` block controlled by a pipeline parameter (`SKIP_TESTS`) so tests can be bypassed for emergency hotfixes. Coverage results are published as an HTML report in Jenkins.

Stage 5 — Docker Build

Builds the image with `--target production` and tags it with two tags:

- `image:BUILD_NUMBER-COMMIT_SHA` — immutable, traceableing tag
- `image:latest` — floating tag for convenience

Stage 6 — Security Scan

Runs **Trivy** (by Aqua Security) against the newly built image to detect known CVEs in OS packages and npm dependencies. Set to `--exit-code 0` so it reports without failing the build — you could change this to `1` to enforce a quality gate.

Stage 7 — Push Image

Uses the `withCredentials` block to securely inject Docker Hub credentials from Jenkins' credential store. The credentials are never hardcoded in the `Jenkinsfile`.

Stage 8 — Deploy

Only runs on the `main` branch. Uses `kubectl apply -k` (Kustomize) to apply the correct environment overlay. It then waits for the rollout to complete with a 120-second timeout before declaring success.

Stage 9 — Smoke Test

A final sanity check: hits the `/health` endpoint on the newly deployed app. If the endpoint doesn't respond, the pipeline is flagged.

Post Actions

- `always` — cleans up the local Docker image and workspace
- `success` — can trigger a Slack or email notification
- `failure` — can trigger an alert

Jenkins Configuration as Code (JCasC)

File: `jenkins/casc.yaml`

Instead of clicking through the Jenkins UI to configure it, everything is declared in a YAML file. This includes:

- Admin user setup
- NodeJS tool installation
- Credential templates

This means Jenkins can be reproduced identically from scratch — no manual setup steps.

---

8. Step 7 — Kubernetes Manifests

Directory: `k8s/base/`

Six manifest files make up the base configuration:

`namespace.yaml`

Creates dedicated namespaces (`todo-dev` and `todo-prod`) to isolate environments within the same cluster.

`configmap.yaml`

Externalizes app configuration (env vars like `NODE_ENV`, `PORT`) from the container image. Changing config no longer requires rebuilding the image.

`deployment.yaml`

The most detailed manifest. Key decisions:

- **Rolling Update strategy** with `maxUnavailable: 0` — zero pods are taken down before a new one is ready, ensuring zero downtime during deploys
- **Three health probes:**
  - `livenessProbe` — Kubernetes restarts the container if `/health` stops responding
  - `readinessProbe` — traffic is only sent to pods where `/ready` returns 200
  - `startupProbe` — gives the container extra time to start before liveness checks begin
- **Resource requests and limits** — prevents one pod from starving others on the node
- **Security context** — runs as non-root, read-only filesystem, all Linux capabilities dropped
- **TopologySpreadConstraints** — spreads pods across different nodes so a single node failure doesn't take the app down

`service.yaml`

A `ClusterIP` service that load-balances traffic across all matching pods. The Ingress routes external traffic to this service.

`ingress.yaml`

Exposes the app outside the cluster via host-based routing (`todo-api.local`). TLS configuration is included but commented out — uncomment and add a cert Secret to enable HTTPS.

`hpa.yaml` **(Horizontal Pod Autoscaler)**

Automatically scales the deployment between 2 and 10 replicas based on:

- CPU utilization > 70%
- Memory utilization > 80%

Scale-up is aggressive (new pods within 30 seconds) while scale-down is conservative (waits 5 minutes to avoid flapping).

---

9. Step 8 — Kustomize Overlays (Dev vs Prod)

Directory: `k8s/overlays/`

Kustomize lets you patch base manifests for different environments **without duplicating YAML**.

Dev overlay (`k8s/overlays/dev/`)

- Targets the `todo-dev` namespace

- Patches the deployment to use **1 replica** (saves resources)
- Sets `NODE_ENV=development`

Prod overlay (`k8s/overlays/prod/`)

- Targets the `todo-prod` namespace
- Patches the deployment to use **3 replicas** (high availability)
- Sets `NODE_ENV=production`

The Jenkins pipeline injects the correct image tag by replacing a placeholder (`IMAGE_TAG_PLACEHOLDER`) in the overlay file with the actual build tag before running `kubectl apply -k`.

---

10. Step 9 — Helper Scripts

Three bash scripts were written to reduce manual steps:

`scripts/setup.sh`

One command to bootstrap the full local environment:

1. Checks that Docker, Node, and npm are installed
2. Runs `docker-compose build && docker-compose up -d`
3. Polls the `/health` endpoint until the app is live
4. Prints a summary of all running services and their URLs
5. Prints the Jenkins initial admin password

`scripts/deploy.sh`

Deploys any image tag to a chosen environment:

```bash
./scripts/deploy.sh dev 42-a3f9b1c
./scripts/deploy.sh prod 42-a3f9b1c
```

It patches the kustomization file, applies it, waits for the rollout, then restores the placeholder for the next deploy.

`scripts/minikube-setup.sh`

Spins up a local Kubernetes cluster using Minikube with the Docker driver, enables the Ingress and Metrics Server addons, and port-forwards the service to `localhost:3000` for local testing.

---

11. Key DevOps Concepts Applied

Concept	How It's Implemented
Infrastructure as Code	All config lives in the repo — no manual server setup
Immutable artifacts	Each build produces a uniquely tagged Docker image
Shift-left testing	Tests run inside the Docker build — a broken image literally cannot be built
GitOps	Kubernetes state is declared in Git; Jenkins applies it
Zero-downtime deploys	Rolling update with `maxUnavailable: 0`
Least privilege	Non-root containers, dropped capabilities, read-only filesystem
Observability	Health/ready/startup probes + Prometheus scrape annotations
Auto-scaling	HPA adjusts replica count based on real load
DRY config	Kustomize overlays patch a single base — no YAML duplication
Secure secrets	Credentials stored in Jenkins credential store, never in code

---

12. Tech Stack Summary

Layer	Technology
Application	Node.js 20, Express 4
Testing	Jest, Supertest
Containerization	Docker (multi-stage), Docker Compose
Reverse Proxy	Nginx
CI/CD	Jenkins (Declarative Pipeline, JCasC)
Image Registry	Docker Hub
Container Security	Trivy (Aqua Security)
Orchestration	Kubernetes
Config Management	Kustomize

Layer	Technology
Auto-scaling	Horizontal Pod Autoscaler (HPA)
Local K8s	Minikube

Built as a DevOps demonstration project covering the full software delivery lifecycle — from code to production.